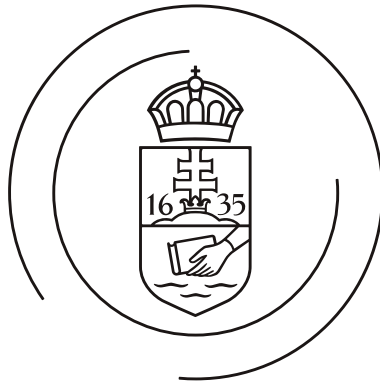




Eötvös Loránd University

ELTE 1

1 Power series

```

1 // POWER SERIES OPERATIONS
2 constexpr int mod = 998244353; // = 2^k * c + 1 | primitív gyöknek jónak kell lennie | 2013265921,
   ↳ 167772161, 2113929217
3 constexpr int N = 1 << 20; // 2^l, l <= k | max N amit transzformálni lehet
4 struct mint {
5     int x;
6     constexpr inline mint(int x = 0) : x(x) {}
7     constexpr inline mint operator+(mint o) const { return x + o.x < mod ? x + o.x : x + o.x - mod; }
8     constexpr inline mint operator-(mint o) const { return x - o.x < 0 ? x - o.x + mod : x - o.x; }
9     constexpr inline mint operator*(mint o) const { return int(uint64_t(x) * o.x % mod); }
10    constexpr inline mint &operator+=(mint o) { return *this = *this + o; }
11    constexpr inline mint &operator-=(mint o) { return *this = *this - o; }
12    constexpr inline mint &operator*=(mint o) { return *this = *this * o; }
13    constexpr inline mint inv() const { return pow(mod - 2); }
14    constexpr inline mint pow(auto x) const {
15        mint a = *this; mint b = 1; for (; x >= 1; x >>= 1) { if (x & 1) { b *= a; } a *= a; } return b;
16    }
17    constexpr inline mint sqrt() const {
18        if (pow(mod >> 1).x != 1) return 0;
19        int Q = (mod - 1) >> (__countr_zero(mod - 1));
20        mint x = pow((Q + 1) >> 1), y = pow(Q);
21        for (int k = __countr_zero(mod - 1) - 1; k >= 0; --k) // TODO: fix 21
22            if (y.pow(1 << k).x != 1) {
23                x *= mint(mod_primitive_root()).pow(mod >> (k + 2));
24                y *= mint(mod_primitive_root()).pow(mod >> (k + 1));
25            }
26        return min(x.x, mod - x.x);
27    }
28    static constexpr long long mod_primitive_root(){ // kiszámítja a moduló egy primitív gyökét
29        long long primes[64] = {}; int size = 0; long long p = 2, m = mod - 1;
30        while (p * p <= m) { if (m % p == 0) primes[size++] = p; while (m % p == 0) m /= p; ++p; } if (m > 1)
   ↳ primes[size++] = m;
31        for (long long i = 2; i < mod; i++) { bool ok = true; for (int j = 0; j < size; j++) ok = ok &&
   ↳ mint(i).pow((mod - 1) / primes[j]).x != 1; if (ok) return i; }
32        return -1;
33    }
34 };
35
36 mint w[N];
37 mint invi[N + 1];
38 __attribute__((constructor)) void init() {
39     invi[1] = w[N / 2] = 1;
40     constexpr mint g = mint(mint::mod_primitive_root()).pow(mod / N);
41     for (int i = N / 2 + 1; i < N; ++i) w[i] = w[i - 1] * g;
42     for (int i = N / 2 - 1; i > 0; --i) w[i] = w[i << 1];
43     for (int i = 2; i <= N; i++) invi[i] = invi[mod % i] * (mint() - mint(mod / i));
44 }
45 void dft(mint f[], int n) { // n kettő hatvány
46     for (int k = n / 2; k; k /= 2)
47         for (int i = 0; i < n; i += k + k)
48             for (int j = 0; j < k; ++j) {
49                 mint x = f[i + j]; mint y = f[i + j + k]; f[i + j] = x + y; f[i + j + k] = (x - y) * w[k +
   ↳ j];
50             }
51 }
52 void ift(mint f[], int n) { // n kettő hatvány
53     for (int k = 1; k < n; k *= 2)
54         for (int i = 0; i < n; i += k + k)
55             for (int j = 0; j < k; ++j) {
56                 mint x = f[i + j]; mint y = f[i + j + k] * w[k + j]; f[i + j] = x + y; f[i + j + k] = x - y;
57             }
58     mint inv = mod - (mod - 1) / n;
59     std::reverse(f + 1, f + n);
60     for (int i = 0; i < n; ++i) f[i] *= inv;
61 }
62 struct poly : std::vector<mint> { using std::vector<mint>::vector;
63     poly &add(const poly &o) { if (size() < o.size()) resize(o.size()); for (int i = 0; i < o.size(); ++i)
   ↳ (*this)[i] += o[i]; return *this; }
64     poly &sub(const poly &o) { if (size() < o.size()) resize(o.size()); for (int i = 0; i < o.size(); ++i)
   ↳ (*this)[i] -= o[i]; return *this; }
65     poly &mul(const poly &o) { if (size() < o.size()) resize(o.size()); for (int i = 0; i < o.size(); ++i)
   ↳ (*this)[i] *= o[i]; return *this; }
66     poly &mul(const mint &o) { for (mint &i: *this) i *= o; return *this; }
67     poly &derivative() { for (int i = 0; i < (int)size() - 1; ++i) (*this)[i] = (*this)[i + 1] * mint(i + 1);
   ↳ pop_back(); return *this; }
68     poly &integral() { resize(size() + 1); for (int i = (int)size() - 1; i > 0; i--) (*this)[i] =
   ↳ (*this)[i - 1] * invi[i]; (*this)[0] = mint(); return *this; } // lehet overflow invi mérete N !!!
69
70     poly copy() const { return *this; }
71     poly &resize(auto sz) { return vector::resize(sz), *this; }
72     poly &dft(int n) { return ::dft(resize(n).data(), n), *this; }
73     poly &ift(int n) { return ::ift(resize(n).data(), n), *this; }
74     poly &ins(int sz) { return insert(begin(), sz, mint()), *this; }
75     poly &del(int sz) { return erase(begin(), begin() + sz), *this; }
76     poly &reverse() { return std::reverse(begin(), end()), *this; }
77     poly pre(int sz) const { return sz < size() ? poly(begin(), begin() + sz) : copy(); }
78     poly &reduce() { while (!empty() && back().x == 0) pop_back(); return *this; }
79
80     poly conv(const poly &o){
81         int n = __bit_ceil(size() + o.size() - 1);

```

```

82     return copy().dft(n).mul(o.copy().dft(n)).ift(n).resize(size() + o.size() - 1);
83 }
84 poly inv() const {
85     if (front().x == 0) return {};
86     int m = size();
87     poly inv = {front().inv()};
88     for (int k = 1; k < m; k *= 2) {
89         int n = k * 2; poly a = inv.copy().dft(n), b = pre(n).dft(n);
90         inv.sub(a.copy().mul(b).ift(n).del(k).dft(n).mul(a).ift(n).resize(k).ins(k));
91     }
92     return inv.resize(m);
93 }
94 poly log() const { // res[0] = 0
95     int n = __bit_ceil(size() * 2 - 1);
96     return copy().derivative().dft(n).mul(inv().dft(n)).ift(n).integral().resize(size());
97 }
98 poly exp() const { // p[0] == 0, különben nem valid az eredmény
99     if (front().x != 0) return {};
100    int m = size();
101    poly e = {1};
102    for (int k = 1; k < m; k *= 2) {
103        int n = k * 2;
104        poly elog = e.resize(n).log(); e.dft(n*2);
105        e.add(pre(n).sub(elog).dft(n*2).mul(e)).ift(n*2).resize(n);
106    }
107    return e.resize(m);
108 }
109 poly pow(auto k) const { // k: int, long long
110     if (k == 0) return poly{1}.resize(size());
111     int j = 0;
112     while (j < size() && (*this)[j].x == 0) ++j;
113     if (j == size()) return poly{0}.resize(size());
114     mint c = (*this)[j];
115     return copy().del(j).mul(c.inv()).log().mul(mint(k % mod)).exp().mul(c.pow(k % (mod - 1))).ins(j >
→ size() / k ? (long long)size() : j * k).resize(size());
116 }
117 poly sqrt() const { // ha nem létezik akkor az eredmény: {}
118     int j = 0;
119     while (j < size() && (*this)[j].x == 0) ++j;
120     if (j == size()) return poly{0}.resize(size());
121     mint c = (*this)[j].sqrt();
122     if (c.x == 0 || j % 2 != 0) return {};
123     return copy().del(j).mul((*this)[j].inv()).resize(size() - j / 2).pow(mint(2).inv().x).mul(c).ins(j
→ / 2);
124 }
125 poly div(const poly& o) {
126     poly a = copy().reduce().reverse(), b = o.copy().reduce().reverse();
127     int m = a.size() - b.size() + 1;
128     if (a.empty() || b.empty() || a.size() < b.size()) return b.empty() ? poly{} : poly{0};
129     return a.conv(b.resize(a.size()).inv()).resize(m).reverse();
130 }
131 poly rem(const poly& o) {
132     return copy().sub(div(o).conv(o));
133 }
134 };
135
136 /*
137 poly: ugyanúgy működik, mint az std::vector
138 műveletek: +, -, * pontoknékt, derivátl, integrál, (resize, dft, ift, ins, del, reverse)
139 konstans műveletek: conv, inv, log, exp, pow, sqrt, div, rem
140 */
141
142 // TREAP
143 mt19937 rnd(42123); // mt19937_64 ha long long kell
144 struct node { // az upd() és push()-t kell implementálni / upd()-et a konstruktor is hívja
145     int val, w, size; // val érték (cserélhető), w súly, size a részfa mérete
146     node *l, *r; // bal, jobb gyerek
147     node(int c) : val(c), w(rnd()), size(1), l(NULL), r(NULL) { upd(); }
148     ~node() { delete l; delete r; }
149     inline void upd() {} // update az l, r-ből
150     inline void push() {} // push l, r-be
151 } *treap;
152 int size(node *p) { return p ? p->size : 0; }
153 void split(node *p, node *&l, node *&r, int val) { // l < val / val <= r
154     if (!p) { l = r = NULL; return; }
155     p->push();
156     if (size(p->l) < val) split(p->r, p->r, r, val - size(p->l) - 1), l = p;
157     else split(p->l, l, p->l, val), r = p;
158     p->size = 1 + size(p->l) + size(p->r); p->upd();
159 }
160 void merge(node *&p, node *l, node *r) {
161     if (!l || !r) { p = l ? l : r; return; }
162     if (l->w < r->w) l->push(), merge(l->r, l->r, r), p = l;
163     else r->push(), merge(r->l, l, r->l), p = r;
164     p->size = 1 + size(p->l) + size(p->r); p->upd();
165 }
166 }

```